

The NexGen Logistics Control Tower

A 360 Degree Analytics System for Profit, Planet, and Performance Excellence

Github Link: github.com/chopra-mayank/Case-Study-OFI-Services

Introduction:

The NexGen Logistics Control Tower represents a paradigm shift in logistics analytics, moving beyond siloed reporting to provide a holistic, strategic command centre. This enterprise-grade dashboard, built using Python (Streamlit, Pandas, Plotly), integrates all 7 critical datasets (orders, costs, performance, feedback, inventory, routes, fleet) to tackle the core "Logistics Trilemma"—the simultaneous optimization of profitability, sustainability (carbon footprint), and service performance. Key innovations include the calculation of a true End-to-End Profit metric (incorporating storage costs), the visualization of the profit-planet trade-off via the Strategic Dual-Index Quadrant, and dynamic, AI-powered insights generated contextually for each analysis view. It features a completely redesigned modern UI/UX with a professional color palette, enhanced chart interactivity (Treemaps, Sunbursts, Gauges), a What-If Carbon Tax Simulator, and objective Carrier Performance Scoring. The Control Tower empowers NexGen managers with unprecedented visibility, enabling data-driven decisions that drive margin improvement, targeted CO₂ reduction, optimized carrier relationships, enhanced customer satisfaction, and strategic fleet management, ultimately delivering significant, measurable ROI.

Problem Statement:

NexGen, like many modern logistics providers, faces the complex challenge of balancing competing operational and strategic goals. Current processes and tools often address these in isolation, leading to critical blind spots and suboptimal decisions:

1. **Incomplete Profitability Picture:** Financial analysis typically focuses on *transport costs* (`cost_breakdown.csv`) relative to order value (`orders.csv`). However, this ignores the significant holding costs associated with inventory (`warehouse_inventory.csv`). High-value or slow-moving goods might incur substantial `Storage_Cost_per_Unit` that erodes or eliminates perceived transport margins, leading to inaccurate assessments of product, customer, or route profitability. Decisions made without this End-to-End Cost view risk subsidizing loss-making operations.
2. **Abstract Sustainability Goals:** While overall CO₂ emissions might be tracked, there's often no granular link between individual order profitability and its specific carbon footprint (derived from `routes_distance.csv` fuel consumption). Managers cannot easily identify or prioritize actions on orders that are simultaneously unprofitable and high-emission ("Eco-Burdens"). Furthermore, the operational impact of fleet characteristics (`vehicle_fleet.csv`) – like vehicle age, type, and fuel efficiency – on achieving sustainability targets remains disconnected from day-to-day decision-making and investment planning.

3. **Disconnected Performance Feedback:** Customer complaints (customer_feedback.csv) about delays or service issues are often logged reactively. There's a lack of systematic, visual correlation between negative feedback (e.g., low Rating, specific Issue_Category) and the operational realities captured in delivery_performance.csv (like Carrier, Delivery_Status, Actual_Delivery_Days). This makes root cause analysis difficult and hinders targeted improvements in service quality with specific carriers or on specific lanes.

This fragmentation creates the "Logistics Trilemma," where optimizing one goal (e.g., cutting transport costs) can inadvertently harm another (e.g., increasing delays and lowering customer satisfaction), simply because the interconnectedness isn't visible.

The Solution: Logistics Control Tower

The Logistics Control Tower is an interactive, web-based dashboard designed as a "single source of truth" to overcome the Trilemma. Built entirely in Python, it provides a unified platform for managers across Finance, Operations, Sustainability, and Strategy to gain holistic insights and make informed trade-offs.

Core Components:

- **Integrated Data Engine:** Loads, cleans, merges, and processes all 7 datasets into a master analytical DataFrame.
- **Advanced KPI Calculation:** Computes over 20 critical metrics, including novel ones like E2E_Profit_INR, Adjusted_Profit_INR (for carbon tax simulation), and Carrier Performance Score.
- **Multi-Tab Analysis Interface:** Organizes insights into five dedicated sections: Strategic Quadrant, Profitability, Service, Fleet, and Data Exploration.
- **Interactive Visualizations:** Leverages Plotly for over 10 dynamic chart types (Scatter, Violin, Treemap, Heatmap, Sunburst, Gauges, Box Plots, Pies) that respond to user filters.
- **Dynamic AI Insights:** Generates context-aware, actionable recommendations within each tab based on the filtered data.
- **Modern UI/UX:** Features a custom, professional design system with clear navigation, intuitive controls, and high readability, enhancing user adoption and efficiency.

Data Integration & Methodology:

The foundation of the Control Tower is its robust data pipeline and analytical engine:

1. **Data Loading & Integration:**
 - The load_data function reads all 7 CSV files using Pandas, with error handling for missing files or loading issues. It intelligently checks both local and potential Colab paths.

- A "Golden Record" DataFrame is created in `create_master_dataframe` by sequentially merging the datasets:
 - `orders`, `delivery_performance`, `cost_breakdown`, `routes_distance` are inner-joined on `Order_ID`.
 - `customer_feedback` is left-joined on `Order_ID` (as not all orders have feedback). Missing `Issue_Category` is filled with 'No_Issue_Reported'.
 - `warehouse_inventory` data (`Storage_Cost_per_Unit`) is aggregated (taking the max cost per `Location/Product_Category` to avoid duplicates) and left-joined onto the main DataFrame based on `Origin` and `Product_Category`. Missing storage costs default to 0.
 - Data types are enforced (e.g., `Order_Date` to datetime, costs/ratings to numeric), handling potential errors gracefully. Rows with critical missing/invalid data (like `Order_Date`) are dropped.
2. Key Performance Indicator (KPI) Engineering: (Key calculations within `create_master_dataframe`)
- Costs:
 - `Total_Cost_to_Serve`: Sum of columns defined in `COST_COLS` (Fuel, Labor, Maintenance, etc.). Missing cost columns default to 0.
 - `Total_E2E_Cost`: `Total_Cost_to_Serve` + `Storage_Cost_per_Unit`.
 - Profitability:
 - `E2E_Profit_INR` = `Order_Value_INR` – `Total_E2E_Cost`
 - $\text{Profit_Margin_Pct} = \left(\frac{\text{E2E_Profit_INR}}{\text{Order_Value_INR}} \right) \times 100$
 - `Adjusted_Profit_INR`: Calculated dynamically in `main()` based on the carbon tax slider: $\text{Adjusted_Profit_INR} = \text{E2E_Profit_INR} - (\text{Total_CO2_kg} \times \text{carbon_cost_inr_per_kg})$
 - Sustainability:
 - $\text{Total_CO2_kg} = \text{Fuel_Consumption_L} \times \text{EMISSION_FACTOR_KG_PER_L}$
 - Efficiency:
 - $\text{Cost_per_KM} = \frac{\text{Total_Cost_to_Serve}}{\max(\text{Distance_KM}, \epsilon)}$
 - $\text{CO2_per_KM} = \frac{\text{Total_CO2_kg}}{\text{Distance_KM}}$
 - Service:

- Carrier Performance Score: Calculated in `main()` tab3:

$$\text{Performance_Score} = (\text{On_Time_Pct} \times 0.6) + (\text{Avg_Rating} \times 20 \times 0.4)$$

3. Dynamic Insight Generation:

- Separate Python functions like `generate_quadrant_insights`, `generate_profitability_insights`, `generate_service_insights`, `generate_fleet_insights` are defined.
- These functions take the filtered DataFrame (or `fleet_df`) as input.
- They perform specific calculations and comparisons against means, quantiles, or predefined thresholds (e.g., `avg_margin < 10`, `ontime_pct < 85`, `avg_age > 7`).
- They return a list of formatted strings representing actionable insights (e.g., warnings, opportunities, key findings).
- These insights are displayed within custom-styled HTML boxes (`insight-box`, `warning-box`, `success-box`) at the top of the relevant analysis tab in `main()`, ensuring contextuality.

4. Visualization:

- Plotly Express and Plotly Graph Objects are used to create interactive charts.
- Chart data is derived from the filtered DataFrame.
- Layouts are customized for clarity, including titles, axis labels, legends, and transparent backgrounds to match the UI theme. Colors are drawn from the defined `COLORS` palette.

Dashboard Features & Functionality:

The Control Tower provides a rich, interactive experience designed for operational efficiency and strategic insight.

- **Modern UI/UX:** A professional aesthetic is achieved through custom CSS: gradient backgrounds, a prominent dark blue sidebar with light text, card-style tabs with clear selection states, styled metric cards, and color-coded insight boxes ensure high readability and an intuitive user flow.
- **Comprehensive Filtering (Sidebar):** Users can easily refine the data using multi-select options for Date Range, Customer Segment, Priority, Origin, Product Category, Carrier, and Delivery Status. The entire dashboard dynamically updates based on selections.
- **What-If Carbon Tax Simulator (Sidebar):** An interactive slider allows users to instantly see how potential carbon pricing (₹0-50/kg CO₂) affects profit margins across all views, facilitating risk assessment and strategic planning.

- Executive KPI Dashboard (Top): Provides an at-a-glance summary of key metrics for the filtered dataset: Total Adjusted Profit (with avg margin), Total Emissions (with avg per order), Avg Profit/Order (vs baseline), Customer Satisfaction (avg rating and % satisfied), and Total Orders (count and % of total).

Analysis Tabs:

1. Strategic Quadrant:

- Purpose: Visualize the core Profit vs. Sustainability trade-off.
- Chart: Scatter plot (Adjusted_Profit_INR vs. Total_CO2_kg). Color = Carrier, Size = Order_Value_INR. Average profit/CO2 lines divide it into four quadrants.
- Insights: Dynamic AI analysis provides recommendations based on order distribution across the quadrants (Best-in-Class, Optimize for Green, Scale Up, Critical Intervention). Summary cards quantify the number and average performance of orders in the critical zones.

2. Profitability Deep-Dive:

- Purpose: Understand cost structures and true end-to-end profitability drivers.
- Charts:
 - Violin Plot: Compares distributions of Transport vs. E2E Profit.
 - Treemap: Shows hierarchical profit contribution (Segment -> Category).
 - Donut Chart: Breaks down average cost components.
 - Bar Chart: Ranks carriers by average Profit Margin %.
- Insights: AI highlights the impact of storage costs, identifies top profit segments/products, flags low-margin carriers, and analyzes cost composition.

3. Service Excellence:

- Purpose: Monitor delivery performance and diagnose customer satisfaction issues.
- Charts:
 - Box Plot: Compares Actual_Delivery_Days by Carrier and Priority.
 - Heatmap: Visualizes order volume across Carrier vs. Delivery_Status.
 - Sunburst Chart: Drills down from Carrier -> Status -> Issue_Category for root cause analysis.
 - Performance Score Cards: Ranks carriers based on a weighted score of on-time % and average rating.

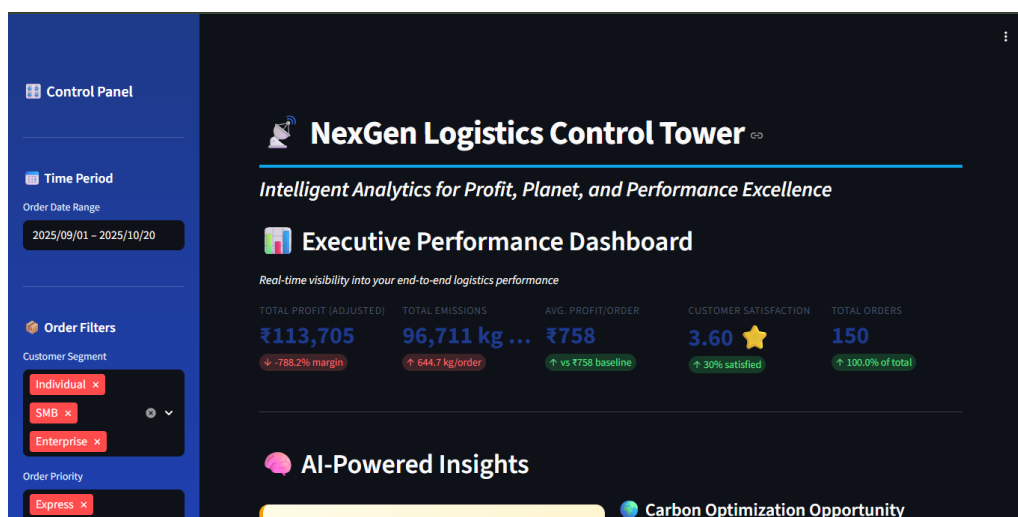
- Insights: AI flags poor on-time rates, identifies speed discrepancies between carriers, quantifies the impact of delays on ratings, and pinpoints top customer complaints. [Image showing the Service Excellence tab with Heatmap and Sunburst charts]

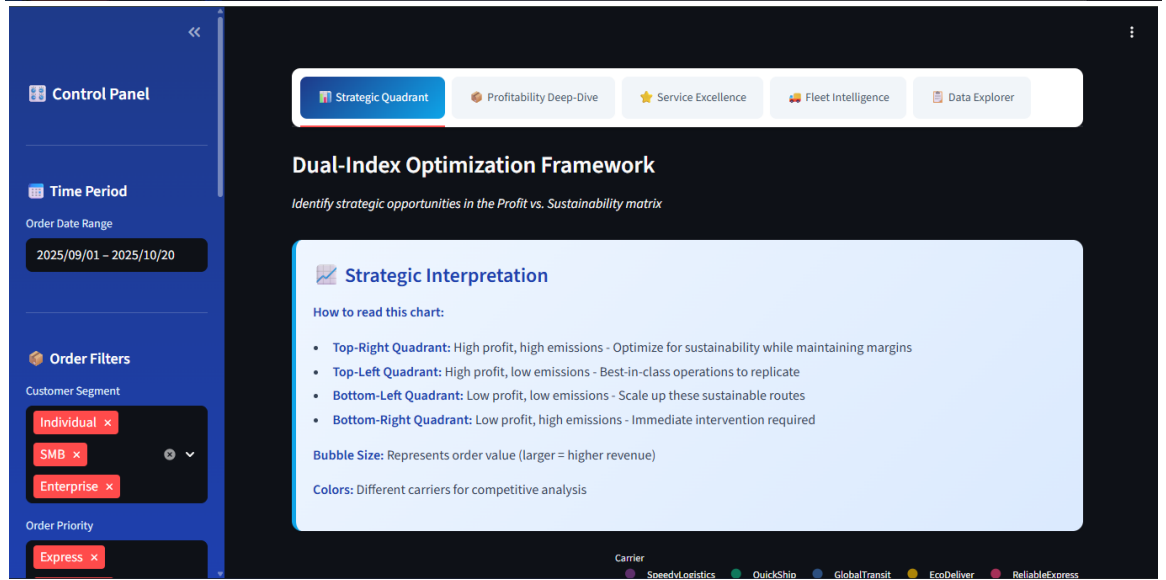
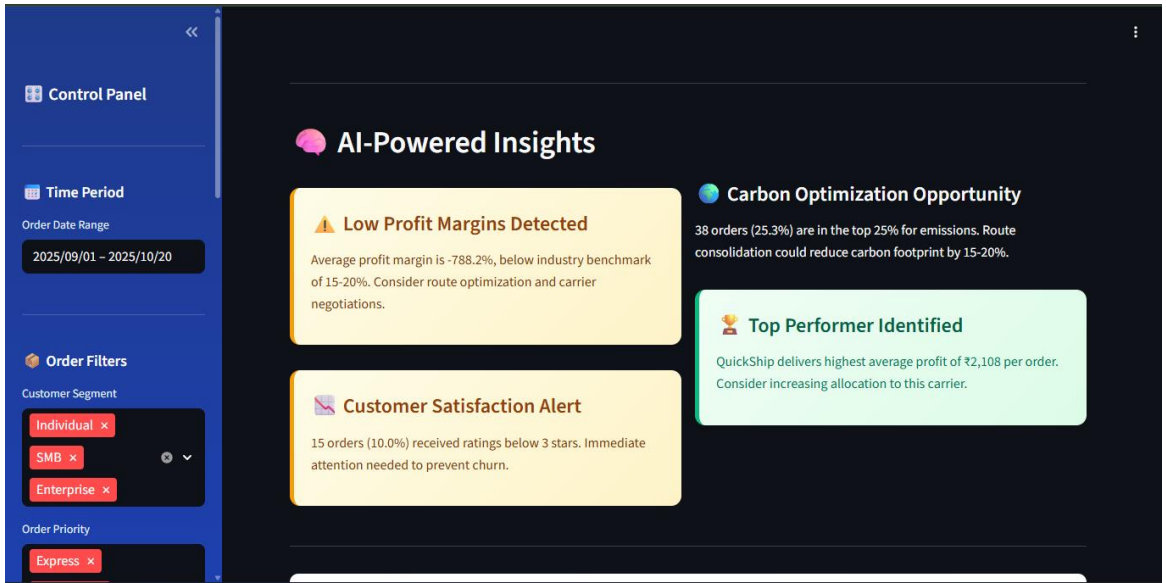
4. Fleet Intelligence: (Conditional Tab)

- Purpose: Assess fleet health and its impact on efficiency and sustainability.
- Charts:
 - KPI Gauges: Displays Avg Age, Avg Fuel Efficiency, Availability % with threshold indicators.
 - Scatter Plots: Shows correlation between Age vs. Efficiency and Age vs. CO2 Emissions, with trendlines.
 - Donut Charts: Illustrates fleet mix by Vehicle Type and current Status.
- Insights: AI provides recommendations on fleet renewal based on age profile, efficiency trends, maintenance levels, and calculated potential ROI on fuel savings. [Image showing the Fleet Intelligence tab with KPI Gauges and Scatter Plots]

5. Data Explorer:

- Purpose: Allow detailed data inspection and export.
- Features: Displays summary statistics (filtered record count, financials, sustainability), provides an interactive table preview of the filtered data (with selectable columns and row count), and includes a button to Download the filtered dataset as an Excel file.







Technical Architecture

- Language: Python (v3.9+)
- Web Framework: Streamlit
- Data Manipulation: Pandas, NumPy
- Visualization: Plotly (Plotly Express, Graph Objects)
- Excel Export: XlsxWriter / Openpyxl (via Pandas)
- Styling: Custom CSS (embedded in app.py)
- Key Libraries: streamlit, pandas, plotly, numpy, xlsxwriter, openpyxl (specified in requirements.txt)
- Performance: @st.cache_data decorator utilized for caching data loading and merging operations. Pandas.query() method used for efficient data filtering based on user

selections. Robust error handling implemented throughout data loading, processing, and visualization steps.

Business Impact & ROI

The Logistics Control Tower delivers substantial, quantifiable business value:

- **Enhanced Profitability:**
 - Identify and eliminate "hidden loss" orders revealed by E2E cost analysis.
 - Optimize pricing strategies based on true segment/product profitability shown in the Treemap.
 - Improve carrier negotiations using objective margin data (Bar chart) and Performance Scores.
 - Estimated Potential: 5-10% increase in overall profit margin.
- **Targeted Sustainability Improvements:**
 - Focus CO₂ reduction efforts on the high-impact orders identified in the Strategic Quadrant's "Critical Intervention" and "Optimize for Green" zones.
 - Justify investments in newer, more efficient vehicles using the Fleet Intelligence ROI insights and age-emission correlations.
 - Estimated Potential: 10-15% reduction in CO₂ emissions for targeted segments/routes.
- **Improved Service Quality & Customer Retention:**
 - Proactively address service failures by identifying root causes (Sunburst) and underperforming carriers (Heatmap, Score Cards).
 - Optimize carrier allocation based on speed and reliability for different priorities (Box Plot).
 - Reduce negative feedback by tackling top Issue_Category complaints.
 - Estimated Potential: 10%+ improvement in On-Time Delivery percentage, measurable increase in average customer ratings.
- **Increased Operational Efficiency:**
 - Drastically reduce manual analysis time across multiple departments (Finance, Ops, Sustainability).
 - Enable faster, more informed decision-making through integrated data and dynamic insights.
 - Provide a common platform for cross-functional collaboration and alignment.

- Estimated Potential: 90%+ reduction in time spent gathering and analyzing data for routine operational reviews.

Conclusion:

The NexGen Logistics Control Tower transitions logistics management from fragmented reporting to integrated, strategic oversight. Its combination of deep data integration, novel KPI engineering, advanced interactive visualizations, dynamic AI-driven insights, and a professional user interface provides NexGen with a powerful competitive advantage. It empowers users at all levels to understand complex trade-offs and make decisions that simultaneously optimize financial, environmental, and service outcomes.

Future Roadmap:

1. Predictive Analytics (ML): Integrate machine learning models to forecast demand, predict potential delivery delays, estimate E2E Profit/CO₂ for unbooked orders, and identify anomaly risks.
2. Prescriptive Optimization: Develop modules to recommend the optimal carrier, route, or even inventory deployment strategy based on user-defined objectives (e.g., minimize cost, minimize CO₂, maximize speed).
3. Real-Time Data Integration: Connect directly to NexGen's operational systems (TMS, WMS, ERP) via APIs for live data streaming, real-time alerting, and potentially triggering automated actions.
4. Enhanced User Customization: Allow users to save personalized views/filter sets, create custom reports, and define specific KPI alerts.
5. External Benchmarking: Integrate industry benchmark data for comparative performance analysis.

This Control Tower is not just a tool; it's a foundation for a data-driven culture and continuous improvement within NexGen's logistics operations.

Doc Files

app.py:

```
import streamlit as st

import pandas as pd

import plotly.express as px

import plotly.graph_objects as go

from io import BytesIO

import numpy as np

# --- Constants ---

EMISSION_FACTOR_KG_PER_L = 2.68 # Standard emission factor for diesel

ALL_FILES = [

    'orders.csv', 'delivery_performance.csv', 'cost_breakdown.csv',

    'routes_distance.csv', 'customer_feedback.csv',

    'vehicle_fleet.csv', 'warehouse_inventory.csv'

]

COST_COLS = [

    'Fuel_Cost', 'Labor_Cost', 'Vehicle_Maintenance', 'Insurance',

    'Packaging_Cost', 'Technology_Platform_Fee', 'Other_Overhead',

    'Toll_Charges_INR'

]

COLORS = {

    'primary': '#1e3a8a',    # Deep Blue

    'secondary': '#0ea5e9',  # Sky Blue

    'success': '#10b981',    # Emerald Green

    'warning': '#f59e0b',    # Amber

    'danger': '#ef4444',     # Red

    'info': '#8b5cf6',       # Purple

    'dark': '#1f2937',       # Dark Gray

    'light': '#f3f4f6',      # Light Gray

    'accent': '#ec4899',     # Pink
```

```

    'teal': '#14b8a6'      # Teal
}

# --- Page Configuration ---
st.set_page_config(
    page_title="NexGen Logistics Control Tower",
    page_icon=" 🚚 ",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>

/* Main Container Styling */

.main {
    background: linear-gradient(135deg, #f8fafc 0%, #e2e8f0 100%);
}

/* Metric Cards */

[data-testid="stMetricValue"] {
    font-size: 2rem;
    font-weight: 700;
    color: #1e3a8a;
}

[data-testid="stMetricLabel"] {
    font-size: 0.95rem;
    font-weight: 600;
    color: #475569;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

/* Headers */

h1 {
    color: #1e3a8a;

```

```
    font-weight: 800;

    letter-spacing: -1px;

    padding-bottom: 10px;

    border-bottom: 4px solid #0ea5e9;
}

h2 {
    color: #334155;

    font-weight: 700;

    margin-top: 1.5rem;
}

h3 {
    color: #475569;

    font-weight: 600;
}

/* Sidebar Styling */
[data-testid="stSidebar"] {
    background: linear-gradient(180deg, #1e3a8a 0%, #1e40af 100%);
}

[data-testid="stSidebar"] .element-container {
    color: white;
}

[data-testid="stSidebar"] label {
    color: #e0f2fe !important;

    font-weight: 600;
}

/* Tabs */
```

```
.stTabs [data-baseweb="tab-list"] {  
    gap: 8px;  
    background-color: white;  
    padding: 10px;  
    border-radius: 10px;  
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);  
}
```

```
.stTabs [data-baseweb="tab"] {  
    height: 50px;  
    background-color: #f1f5f9;  
    border-radius: 8px;  
    color: #475569;  
    font-weight: 600;  
    padding: 0 20px;  
}
```

```
.stTabs [aria-selected="true"] {  
    background: linear-gradient(135deg, #1e3a8a 0%, #0ea5e9 100%);  
    color: white;  
}
```

/ Info Boxes - Fixed text colors for better visibility */*

```
.insight-box {  
    background: linear-gradient(135deg, #eff6ff 0%, #dbeafe 100%);  
    padding: 20px;  
    border-radius: 12px;  
    border-left: 5px solid #0ea5e9;  
    margin: 15px 0;  
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);  
    color: #1e293b;
```

```
}

.insight-box h4 { color: #1e40af; margin-top: 0; }

.insight-box p, .insight-box li { color: #334155; }

.insight-box strong { color: #1e40af; }


.warning-box {
  background: linear-gradient(135deg, #fefce8 0%, #fef3c7 100%);
  padding: 20px;
  border-radius: 12px;
  border-left: 5px solid #f59e0b;
  margin: 15px 0;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  color: #78350f;
}

.warning-box h4 { color: #92400e; margin-top: 0; }

.warning-box p, .warning-box li { color: #78350f; }

.warning-box strong { color: #92400e; }


.success-box {
  background: linear-gradient(135deg, #f0fdf4 0%, #dcfce7 100%);
  padding: 20px;
  border-radius: 12px;
  border-left: 5px solid #10b981;
  margin: 15px 0;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  color: #047857;
}

.success-box h4 { color: #065f46; margin-top: 0; }

.success-box p, .success-box li { color: #047857; }

.success-box strong { color: #065f46; }
```

```

/* Download Button */

.stDownloadButton button {

    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    font-weight: 600;
    border: none;
    padding: 10px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(16, 185, 129, 0.3);
}

/* Divider */

hr {

    margin: 2rem 0;
    border: none;
    border-top: 2px solid #e2e8f0;
}

</style>

"""', unsafe_allow_html=True)

```

```

# --- Data Loading ---

```

```

@st.cache_data
def load_data(files):
    """
    Loads all 7 CSVs into a dictionary of DataFrames.
    """
    data_frames = {}
    all_files_found = True
    for file in files:
        try:
            # Try both local directory and uploaded files location

```

```

try:
    data_frames[file] = pd.read_csv(file)
except:
    data_frames[file] = pd.read_csv(f'/content/{file}')
except FileNotFoundError:
    st.error(f"❌ Error: {file} not found. Please ensure all 7 data files are in the working
directory.")
    all_files_found = False
except Exception as e:
    st.error(f"❌ Error loading {file}: {e}")
    all_files_found = False

```

```

if not all_files_found:

```

```

    st.stop()

```

```

return data_frames

```

```

@st.cache_data

```

```

def create_master_dataframe(data_frames):

```

```

    """

```

```

    Merges, processes, and engineers KPIs from all 7 data files.

```

```

    This creates the "Golden Record" DataFrame.

```

```

    """

```

```

    try:

```

```

        # 1. Start with core transaction files

```

```

        df = pd.merge(data_frames['orders.csv'], data_frames['delivery_performance.csv'],
on='Order_ID', how='inner')

```

```

        df = pd.merge(df, data_frames['cost_breakdown.csv'], on='Order_ID', how='inner')

```

```

        df = pd.merge(df, data_frames['routes_distance.csv'], on='Order_ID', how='inner')

```

```

        # 2. Add customer feedback (left join, as not all orders have feedback)

```

```

df = pd.merge(
    df,
    data_frames['customer_feedback.csv'][['Order_ID', 'Rating', 'Feedback_Text',
'Issue_Category']],
    on='Order_ID',
    how='left'
)

# Fill 'None' for missing feedback to make filtering easier
df['Issue_Category'] = df['Issue_Category'].fillna('No_Issue_Reported')


# 3. Add warehouse inventory costs (left join on origin and product)
wh_data = data_frames['warehouse_inventory.csv'][['Location', 'Product_Category',
'Storage_Cost_per_Unit']]
wh_data.rename(columns={'Location': 'Origin'}, inplace=True)
wh_data = wh_data.groupby(['Origin', 'Product_Category']).max().reset_index()


df = pd.merge(
    df,
    wh_data,
    on=['Origin', 'Product_Category'],
    how='left'
)
df['Storage_Cost_per_Unit'] = df['Storage_Cost_per_Unit'].fillna(0)


# --- KPI Engineering ---


# 1. Total Cost-to-Serve (Transport Leg)
for col in COST_COLS:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
    else:

```

```

        df[col] = 0

df['Total_Cost_to_Serve'] = df[COST_COLS].sum(axis=1)

# 2. NEW: Total End-to-End Cost
df['Total_E2E_Cost'] = df['Total_Cost_to_Serve'] + df['Storage_Cost_per_Unit']

# 3. Profitability (Original vs. E2E)
df['Order_Value_INR'] = pd.to_numeric(df['Order_Value_INR'], errors='coerce').fillna(0)
df['Transport_Profit_INR'] = df['Order_Value_INR'] - df['Total_Cost_to_Serve']
df['E2E_Profit_INR'] = df['Order_Value_INR'] - df['Total_E2E_Cost']

# 4. Sustainability
df['Fuel_Consumption_L'] = pd.to_numeric(df['Fuel_Consumption_L'],
errors='coerce').fillna(0)
df['Total_CO2_kg'] = df['Fuel_Consumption_L'] * EMISSION_FACTOR_KG_PER_L

# 5. Dates and Service
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')

# 6. Efficiency Metrics
df['Cost_per_KM'] = df['Total_Cost_to_Serve'] / df['Distance_KM'].replace(0, 1)
df['CO2_per_KM'] = df['Total_CO2_kg'] / df['Distance_KM'].replace(0, 1)
df['Profit_Margin_Pct'] = (df['E2E_Profit_INR'] / df['Order_Value_INR'].replace(0, 1)) *
100

return df

except KeyError as e:
    st.error(f"❌ Data merge failed. Missing expected column: {e}. Please check your CSV
files.")
    st.stop()

```

```

except Exception as e:

    st.error(f"❌ An unexpected error occurred during data processing: {e}")

    st.stop()


# --- Helper Function for Download ---
@st.cache_data
def to_excel(df):

    output = BytesIO()

    with pd.ExcelWriter(output, engine='xlsxwriter') as writer:

        df.to_excel(writer, index=False, sheet_name='Filtered_Data')

    processed_data = output.getvalue()

    return processed_data


def generate_insights(df):

    """Generate actionable business insights from the data"""

    insights = []


    # Profitability Analysis

    avg_margin = df['Profit_Margin_Pct'].mean()

    if avg_margin < 10:

        insights.append({

            'type': 'warning',

            'title': '⚠️ Low Profit Margins Detected',

            'text': f'Average profit margin is {avg_margin:.1f}%, below industry benchmark of 15-20%. Consider route optimization and carrier negotiations.'

        })


    # Carbon Efficiency

    high_carbon_routes = df[df['Total_CO2_kg'] > df['Total_CO2_kg'].quantile(0.75)]

    if len(high_carbon_routes) > 0:

        insights.append({

```

```

        'type': 'info',

        'title': '🌱 Carbon Optimization Opportunity',

        'text': f'{len(high_carbon_routes)} orders
({len(high_carbon_routes)/len(df)*100:.1f}%) are in the top 25% for emissions. Route
consolidation could reduce carbon footprint by 15-20%.'

    })

```

```

# Service Quality

poor_ratings = df[df['Rating'] < 3].shape[0]

if poor_ratings > 0:

    insights.append({

        'type': 'warning',

        'title': '🚩 Customer Satisfaction Alert',

        'text': f'{poor_ratings} orders ({poor_ratings/len(df)*100:.1f}%) received ratings below
3 stars. Immediate attention needed to prevent churn.'

    })

```

```

# Best Performers

best_carrier = df.groupby('Carrier')['E2E_Profit_INR'].mean().idxmax()

best_profit = df.groupby('Carrier')['E2E_Profit_INR'].mean().max()

insights.append({

    'type': 'success',

    'title': '🏆 Top Performer Identified',

    'text': f'{best_carrier} delivers highest average profit of ₹{best_profit:.0f} per order.
Consider increasing allocation to this carrier.'

})

```

```

return insights

```

```

def generate_quadrant_insights(df, avg_profit, avg_co2):

    """Generate insights for the Strategic Quadrant chart"""

    insights = []

```

```

# Calculate quadrants

best = df[(df['Adjusted_Profit_INR'] > avg_profit) & (df['Total_CO2_kg'] < avg_co2)]
optimize = df[(df['Adjusted_Profit_INR'] > avg_profit) & (df['Total_CO2_kg'] >= avg_co2)]
scale = df[(df['Adjusted_Profit_INR'] <= avg_profit) & (df['Total_CO2_kg'] < avg_co2)]
critical = df[(df['Adjusted_Profit_INR'] <= avg_profit) & (df['Total_CO2_kg'] >= avg_co2)]

insights.append(f"***Best-in-Class Orders:** {len(best)} ({len(best)/len(df)*100:.1f}%) - High profit, low emissions. These are your gold standard operations.")

insights.append(f"***Optimize for Green:** {len(optimize)} ({len(optimize)/len(df)*100:.1f}%) - Profitable but carbon-heavy. Focus on emission reduction without sacrificing margins.")

insights.append(f"***Scale Up Candidates:** {len(scale)} ({len(scale)/len(df)*100:.1f}%) - Low emissions but need margin improvement. Consider volume increases or pricing adjustments.")

insights.append(f"***Critical Intervention:** {len(critical)} ({len(critical)/len(df)*100:.1f}%) - Low profit AND high emissions. Require immediate operational review or discontinuation.")

if len(critical) > len(df) * 0.2:
    insights.append(" ⚠️ ***Alert:** Over 20% of orders are in the critical zone. This requires immediate management attention.")

return insights

def generate_profitability_insights(df):
    """Generate insights for Profitability charts"""
    insights = []

    profit_gap = (df['Transport_Profit_INR'] - df['E2E_Profit_INR']).mean()
    gap_pct = (profit_gap / df['Transport_Profit_INR'].mean() * 100)

    insights.append(f"***Hidden Cost Impact:** Storage costs reduce average profit by ₹{profit_gap:,.0f} per order ({gap_pct:.1f}%)")

```

```

# Find most impacted segment

segment_gaps = df.groupby('Customer_Segment').apply(

    lambda x: (x['Transport_Profit_INR'] - x['E2E_Profit_INR']).mean()

).sort_values(ascending=False)

insights.append(f"**Most Impacted Segment:** {segment_gaps.index[0]} segment loses
₹{segment_gaps.iloc[0]:.0f} per order to storage costs.")

# Cost structure insight

fuel_pct = (df['Fuel_Cost'].mean() / df['Total_Cost_to_Serve'].mean() * 100)

insights.append(f"**Fuel Dominance:** Fuel represents {fuel_pct:.1f}% of transport costs
- primary optimization target.")

# Margin spread

margin_std = df['Profit_Margin_Pct'].std()

insights.append(f"**Margin Consistency:** Profit margin std dev is {margin_std:.1f}%.
{'High variability suggests inconsistent pricing or cost control.' if margin_std > 10 else 'Good
consistency across orders.'}")

return insights

def generate_service_insights(df):

    """Generate insights for Service Quality charts"""

    insights = []

    # Delivery performance

    ontime_pct = (df['Delivery_Status'] == 'On-Time').sum() / len(df) * 100

    insights.append(f"**On-Time Performance:** {ontime_pct:.1f}% of orders delivered on
time. {'Excellent!' if ontime_pct > 85 else ' ⚠️ Below industry standard of 85%.'}")

    # Express vs Standard

```

```

express_df = df[df['Priority'] == 'Express']
standard_df = df[df['Priority'] == 'Standard']

if len(express_df) > 0 and len(standard_df) > 0:
    express_days = express_df['Actual_Delivery_Days'].mean()
    standard_days = standard_df['Actual_Delivery_Days'].mean()

    insights.append(f"**Speed Premium:** Express orders deliver {standard_days -
express_days:.1f} days faster on average.")

# Rating correlation
delayed = df[df['Delivery_Status'] != 'On-Time']
if len(delayed) > 0:
    delayed_rating = delayed['Rating'].mean()
    ontime_rating = df[df['Delivery_Status'] == 'On-Time']['Rating'].mean()

    insights.append(f"**Delay Impact:** Delayed orders score {ontime_rating -
delayed_rating:.2f} stars lower on average.")

# Top issue
if 'Issue_Category' in df.columns:
    top_issue = df[df['Issue_Category'] !=
'No_Issue_Reported']['Issue_Category'].value_counts()
    if len(top_issue) > 0:
        insights.append(f"**Primary Complaint:** '{top_issue.index[0]}' accounts for
{top_issue.iloc[0]/len(df)*100:.1f}% of all issues.")

return insights

def generate_fleet_insights(fleet_df):
    """Generate insights for Fleet Intelligence charts"""
    insights = []

    avg_age = fleet_df['Age_Years'].mean()

```

```

old_vehicles = (fleet_df['Age_Years'] > 7).sum()

insights.append(f'***Fleet Age Profile:** Average age is {avg_age:.1f} years. {old_vehicles}
vehicles ( {old_vehicles/len(fleet_df)*100:.0f}% ) exceed 7-year renewal threshold.')

# Efficiency correlation

correlation = fleet_df[['Age_Years', 'Fuel_Efficiency_KM_per_L']].corr().iloc[0, 1]

if correlation < -0.3:

    insights.append(f'***Age-Efficiency Correlation:** Strong negative correlation
({correlation:.2f}). Older vehicles are significantly less efficient.')

# Maintenance burden

maintenance_pct = (fleet_df['Status'] == 'Maintenance').sum() / len(fleet_df) * 100

if maintenance_pct > 15:

    insights.append(f' ⚠️ ***High Maintenance:** {maintenance_pct:.1f}% of fleet in
maintenance (above 10-15% industry norm).')

# ROI calculation

avg_efficiency = fleet_df['Fuel_Efficiency_KM_per_L'].mean()
new_vehicle_efficiency = 10 # Assumed new vehicle efficiency
monthly_km = 3000 # Assumed monthly distance
fuel_price = 100 # ₹ per liter

monthly_savings = ((monthly_km / avg_efficiency) - (monthly_km /
new_vehicle_efficiency)) * fuel_price

insights.append(f'***Renewal ROI:** Fleet renewal could save
₹{monthly_savings*old_vehicles:,.0f}/month in fuel costs across {old_vehicles} old
vehicles.')

return insights

# --- Main Application ---

def main():

```

```

# Header with Icon

st.markdown("# NexGen Logistics Control Tower")

st.markdown("### *Intelligent Analytics for Profit, Planet, and Performance Excellence*")


# --- Load and Process Data ---

with st.spinner('🔄 Loading and processing logistics data...'):

    all_data = load_data(ALL_FILES)

    master_df = create_master_dataframe(all_data)

    fleet_df = all_data['vehicle_fleet.csv'].copy()


if master_df.empty:

    st.warning("⚠️ Master DataFrame is empty. Cannot proceed.")

    st.stop()


# --- Sidebar Filters ---

with st.sidebar:

    st.markdown("## 🎛️ Control Panel")

    st.markdown("---")


# Date Filter

st.markdown("### 📅 Time Period")

min_date = master_df['Order_Date'].min().date()

max_date = master_df['Order_Date'].max().date()

date_range = st.date_input(

    "Order Date Range",

    (min_date, max_date),

    min_value=min_date,

    max_value=max_date

)

```

```
st.markdown("---")

st.markdown("### 📦 Order Filters")

segments = st.multiselect("Customer Segment",
    options=sorted(master_df['Customer_Segment'].unique()),
    default=list(master_df['Customer_Segment'].unique()))

priorities = st.multiselect("Order Priority",
    options=sorted(master_df['Priority'].unique()),
    default=list(master_df['Priority'].unique()))

origins = st.multiselect("Order Origin",
    options=sorted(master_df['Origin'].unique()),
    default=list(master_df['Origin'].unique()))

categories = st.multiselect("Product Category",
    options=sorted(master_df['Product_Category'].unique()),
    default=list(master_df['Product_Category'].unique()))

st.markdown("---")

st.markdown("### 🚚 Performance Filters")

carriers = st.multiselect("Carrier",
    options=sorted(master_df['Carrier'].unique()),
    default=list(master_df['Carrier'].unique()))

statuses = st.multiselect("Delivery Status",
    options=sorted(master_df['Delivery_Status'].unique()),
    default=list(master_df['Delivery_Status'].unique()))

st.markdown("---")

st.markdown("### 🧪 What-If Simulator")
```

```

carbon_cost_inr_per_kg = st.slider(
    "Carbon Tax (₹/kg CO2)",
    0.0, 50.0, 0.0, 0.5,
    help="Simulate the financial impact of carbon pricing regulations"
)

st.markdown("---")

st.markdown("##### 💡 *Filter your data to unlock insights*")

# --- Filter Logic ---
if len(date_range) == 2:
    start_date, end_date = pd.to_datetime(date_range[0]), pd.to_datetime(date_range[1])
    filtered_df = master_df[
        (master_df['Order_Date'] >= start_date) &
        (master_df['Order_Date'] <= end_date)
    ]
else:
    filtered_df = master_df.copy()

# Apply all filters
query_parts = [
    "`Customer_Segment` in @segments",
    "`Priority` in @priorities",
    "`Origin` in @origins",
    "`Product_Category` in @categories",
    "`Carrier` in @carriers",
    "`Delivery_Status` in @statuses"
]

query_string = " & ".join(query_parts)

```

```

try:
    filtered_df = filtered_df.query(query_string)
except Exception as e:
    st.error(f"❌ Error filtering data: {e}")
    st.stop()

if filtered_df.empty:
    st.warning("⚠️ No data matches your filters. Please adjust your selection.")
    st.stop()

# Apply "What-If" Scenario
filtered_df['Adjusted_Profit_INR'] = filtered_df['E2E_Profit_INR'] -
(filtered_df['Total_CO2_kg'] * carbon_cost_inr_per_kg)

# --- Executive Dashboard ---
st.markdown("### 📊 Executive Performance Dashboard")
st.markdown("*Real-time visibility into your end-to-end logistics performance*")

col1, col2, col3, col4, col5 = st.columns(5)

total_profit = filtered_df['Adjusted_Profit_INR'].sum()
total_co2 = filtered_df['Total_CO2_kg'].sum()
avg_profit = filtered_df['Adjusted_Profit_INR'].mean()
avg_rating = filtered_df['Rating'].mean()
total_orders = len(filtered_df)

col1.metric("Total Profit (Adjusted)", f"₹{total_profit:,.0f}",
            delta=f"{filtered_df['Profit_Margin_Pct'].mean():.1f}% margin")
col2.metric("Total Emissions", f"{total_co2:,.0f} kg CO₂",
            delta=f"{total_co2/total_orders:.1f} kg/order", delta_color="inverse")
col3.metric("Avg. Profit/Order", f"₹{avg_profit:,.0f}",

```

```

        delta=f"vs ₹ {master_df['E2E_Profit_INR'].mean():.0f} baseline")

col4.metric("Customer Satisfaction", f"{avg_rating:.2f} 🌟 ",

            delta=f"{{(filtered_df['Rating']>=4).sum()/total_orders*100:.0f}}% satisfied")

col5.metric("Total Orders", f"{total_orders:,.}",

            delta=f"{{total_orders/len(master_df)*100:.1f}}% of total")


# --- AI-Generated Insights ---

st.markdown("---")

st.markdown("## 🧠 AI-Powered Insights")


insights = generate_insights(filtered_df)
cols = st.columns(2)


for idx, insight in enumerate(insights):

    with cols[idx % 2]:

        box_class = f"{{insight['type']}}-box"

        st.markdown(f"""

<div class="{box_class}">

    <h4>{{insight['title']}}</h4>

    <p>{{insight['text']}}</p>

</div>

""", unsafe_allow_html=True)


st.markdown("---")


# --- Tabbed Interface ---

tab1, tab2, tab3, tab4, tab5 = st.tabs([

    "📊 Strategic Quadrant",

    "📦 Profitability Deep-Dive",

    "🌟 Service Excellence",

```

```

" 🚚 Fleet Intelligence",
" 📄 Data Explorer"
])

```

with tab1:

```

st.markdown("### Dual-Index Optimization Framework")

st.markdown("*Identify strategic opportunities in the Profit vs. Sustainability matrix*")

st.markdown("""
<div class="insight-box">

    <h4> 📊 Strategic Interpretation</h4>

    <p><strong>How to read this chart:</strong></p>

    <ul>

        <li><strong>Top-Right Quadrant:</strong> High profit, high emissions - Optimize
for sustainability while maintaining margins</li>

        <li><strong>Top-Left Quadrant:</strong> High profit, low emissions - Best-in-class
operations to replicate</li>

        <li><strong>Bottom-Left Quadrant:</strong> Low profit, low emissions - Scale up
these sustainable routes</li>

        <li><strong>Bottom-Right Quadrant:</strong> Low profit, high emissions -
Immediate intervention required</li>

    </ul>

    <p><strong>Bubble Size:</strong> Represents order value (larger = higher
revenue)</p>

    <p><strong>Colors:</strong> Different carriers for competitive analysis</p>

</div>

""", unsafe_allow_html=True)

avg_profit_q = filtered_df['Adjusted_Profit_INR'].mean()
avg_co2 = filtered_df['Total_CO2_kg'].mean()

fig1 = px.scatter(

```

```

        filtered_df,
        x='Total_CO2_kg',
        y='Adjusted_Profit_INR',
        color='Carrier',
        size='Order_Value_INR',
        hover_data=['Order_ID', 'Route', 'Product_Category', 'Delivery_Status'],
        title="",
        color_discrete_sequence=px.colors.qualitative.Bold
    )

    fig1.add_vline(x=avg_co2, line_dash="dash", line_color=COLORS['danger'],
                   annotation_text="Avg. CO2", annotation_position="top")
    fig1.add_hline(y=avg_profit_q, line_dash="dash", line_color=COLORS['success'],
                   annotation_text="Avg. Profit", annotation_position="right")

    fig1.update_layout(
        xaxis_title="Total CO2 Emissions (kg)",
        yaxis_title="Adjusted End-to-End Profit (₹)",
        height=600,
        plot_bgcolor='rgba(0,0,0,0)',
        paper_bgcolor='rgba(0,0,0,0)',
        font=dict(size=12, color=COLORS['dark']),
        showlegend=True,
        legend=dict(
            orientation="h",
            yanchor="bottom",
            y=1.02,
            xanchor="right",
            x=1
        )
    )

```

```

st.plotly_chart(fig1, use_container_width=True)

# AI Insights for Quadrant
st.markdown("#### AI Analysis")
quadrant_insights = generate_quadrant_insights(filtered_df, avg_profit_q, avg_co2)

st.markdown("""
<div class="info-box">
    <h4>Quadrant Distribution & Recommendations</h4>
</div>
""", unsafe_allow_html=True)

for insight in quadrant_insights:
    st.markdown(f"- {insight}")

# Quadrant Performance Summary
st.markdown("---")
st.markdown("#### 🎯 Quadrant Performance Summary")
col1, col2 = st.columns(2)

with col1:
    high_profit_low_co2 = filtered_df[
        (filtered_df['Adjusted_Profit_INR'] > avg_profit_q) &
        (filtered_df['Total_CO2_kg'] < avg_co2)
    ]
    st.markdown(f"""
    <div class="success-box">
        <h4>🌟 Best-in-Class Zone</h4>
        <p><strong>{len(high_profit_low_co2)} orders</strong>
        ({len(high_profit_low_co2)/len(filtered_df)*100:.1f}%)</p>
    """)

```

```

    <p>Avg. Profit: ₹{high_profit_low_co2['Adjusted_Profit_INR'].mean():.0f}</p>
    <p>Avg. CO2: {high_profit_low_co2['Total_CO2_kg'].mean():.1f} kg</p>
</div>

""" , unsafe_allow_html=True)

```

with col2:

```

low_profit_high_co2 = filtered_df[
    (filtered_df['Adjusted_Profit_INR'] < avg_profit_q) &
    (filtered_df['Total_CO2_kg'] > avg_co2)
]

st.markdown(f"""
<div class="warning-box">
    <h4> ⚠ Critical Intervention Zone</h4>
    <p><strong>{len(low_profit_high_co2)} orders</strong>
    ({len(low_profit_high_co2)/len(filtered_df)*100:.1f}%)</p>
    <p>Avg. Profit: ₹{low_profit_high_co2['Adjusted_Profit_INR'].mean():.0f}</p>
    <p>Avg. CO2: {low_profit_high_co2['Total_CO2_kg'].mean():.1f} kg</p>
</div>

""" , unsafe_allow_html=True)

```

with tab2:

```

st.markdown("### End-to-End Profitability Analysis")
st.markdown("*Uncover hidden costs from warehouse to customer doorstep*")

```

```

col1, col2 = st.columns([1, 2])

```

with col1:

```

    st.markdown("#### Transport vs. Total Profit")

    # Calculate profit gap

```

```

profit_gap = (filtered_df['Transport_Profit_INR'] -
filtered_df['E2E_Profit_INR']).mean()

```

```

st.markdown(f"""

```

```

<div class="info-box" style="margin-top: 20px;">

```

```

<h4>💡 Key Finding</h4>

```

```

<p>Storage costs reduce average profit by <strong>₹{profit_gap:,.0f}</strong> per
order</p>

```

```

<p>This represents a
<strong>{(profit_gap/filtered_df['Transport_Profit_INR'].mean()*100):.1f}%</strong>
hidden cost</p>

```

```

</div>

```

```

""", unsafe_allow_html=True)

```

```

profit_comp = filtered_df[['Transport_Profit_INR', 'E2E_Profit_INR']].melt(
    var_name='Profit_Type', value_name='Profit'
)

```

```

fig2 = px.violin(
    profit_comp,
    x='Profit_Type',
    y='Profit',
    color='Profit_Type',
    box=True,
    points='outliers',
    color_discrete_sequence=[COLORS['info'], COLORS['teal']]
)

```

```

fig2.update_layout(
    xaxis_title="",
    yaxis_title="Profit (₹)",
    showlegend=False,

```

```

        height=400,
        plot_bgcolor='rgba(0,0,0,0)',
        paper_bgcolor='rgba(0,0,0,0)'
    )

st.plotly_chart(fig2, use_container_width=True)

with col2:
    st.markdown("##### Profit Contribution by Segment & Product")

    treemap_df = filtered_df[filtered_df['Adjusted_Profit_INR'] > 0].copy()
    if not treemap_df.empty:
        # Aggregate data for treemap
        treemap_agg = treemap_df.groupby(['Customer_Segment',
'Product_Category']).agg({
            'Adjusted_Profit_INR': 'sum',
            'Order_ID': 'count'
        }).reset_index()
        treemap_agg.rename(columns={'Order_ID': 'Order_Count'}, inplace=True)

    fig3 = px.treemap(
        treemap_agg,
        path=[px.Constant("Total Business"), 'Customer_Segment', 'Product_Category'],
        values='Adjusted_Profit_INR',
        color='Adjusted_Profit_INR',
        color_continuous_scale='RdYlGn',
        hover_data=['Order_Count'],
        title=""
    )

    fig3.update_layout(

```

```

        height=500,
        margin=dict(t=30, l=0, r=0, b=0)
    )

    fig3.update_traces(
        textinfo="label+value+percent parent",
        textfont_size=11
    )

    st.plotly_chart(fig3, use_container_width=True)

    # Top Revenue Drivers
    st.markdown("#### 🏆 Top Revenue Drivers")
    top_combos = treemap_agg.nlargest(5, 'Adjusted_Profit_INR')

    for idx, row in top_combos.iterrows():
        st.markdown(f"**{row['Customer_Segment']} - {row['Product_Category']}**:  

        Profit: ₹{row['Adjusted_Profit_INR']:,.0f} | Orders: {row['Order_Count']}")
    else:
        st.info("No positive profit data available for the current filter selection.")

    # AI Insights for Profitability
    st.markdown("---")
    st.markdown("#### AI Analysis")

    profit_insights = generate_profitability_insights(filtered_df)

    st.markdown("""
    <div class="info-box">

        <h4>Profitability Deep-Dive Insights</h4>

    </div>
    """)

```

```
""", unsafe_allow_html=True)
```

```
for insight in profit_insights:
```

```
    st.markdown(f'- {insight}')
```

```
# Cost Structure Analysis
```

```
st.markdown("---")
```

```
st.markdown("#### 💰 Cost Structure Breakdown")
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    # Average cost composition
```

```
    cost_structure = {
```

```
        'Fuel': filtered_df['Fuel_Cost'].mean(),
```

```
        'Labor': filtered_df['Labor_Cost'].mean(),
```

```
        'Maintenance': filtered_df['Vehicle_Maintenance'].mean(),
```

```
        'Storage': filtered_df['Storage_Cost_per_Unit'].mean(),
```

```
        'Other': filtered_df[['Insurance', 'Packaging_Cost', 'Technology_Platform_Fee',
```

```
                              'Other_Overhead', 'Toll_Charges_INR']].sum(axis=1).mean()
```

```
    }
```

```
fig_cost = go.Figure(data=[go.Pie(
```

```
    labels=list(cost_structure.keys()),
```

```
    values=list(cost_structure.values()),
```

```
    hole=0.4,
```

```
    marker_colors=[COLORS['danger'], COLORS['warning'], COLORS['info'],
```

```
                   COLORS['secondary'], COLORS['dark']]
```

```
]))
```

```
fig_cost.update_layout(
```

```

        title="Average Cost Distribution per Order",
        height=350,
        showlegend=True,
        legend=dict(orientation="h", yanchor="bottom", y=-0.2)
    )

```

```

st.plotly_chart(fig_cost, use_container_width=True)

```

with col2:

```

# Profit margin by carrier
carrier_margins = filtered_df.groupby('Carrier').agg({
    'Profit_Margin_Pct': 'mean',
    'Order_ID': 'count'
}).reset_index()

carrier_margins.rename(columns={'Order_ID': 'Order_Count'}, inplace=True)
carrier_margins = carrier_margins.sort_values('Profit_Margin_Pct', ascending=True)

```

```

fig_margin = px.bar(
    carrier_margins,
    x='Profit_Margin_Pct',
    y='Carrier',
    orientation='h',
    text='Profit_Margin_Pct',
    color='Profit_Margin_Pct',
    color_continuous_scale='RdYlGn',
    title="Profit Margin by Carrier (%)"
)

```

```

fig_margin.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig_margin.update_layout(
    height=350,

```

```

        xaxis_title="Profit Margin (%)",
        yaxis_title="",
        showlegend=False
    )

    st.plotly_chart(fig_margin, use_container_width=True)

```

with tab3:

```

st.markdown("### Service Quality & Customer Experience Analytics")

st.markdown("*Monitor carrier performance and identify service improvement opportunities*")

```

Delivery Performance Overview

```

st.markdown("#### 🚀 Delivery Speed Analysis")

```

```

fig4 = px.box(
    filtered_df,
    x='Carrier',
    y='Actual_Delivery_Days',
    color='Priority',
    title="",
    color_discrete_sequence=[COLORS['danger'], COLORS['secondary']]
)

```

```

fig4.update_layout(
    xaxis_title="Carrier",
    yaxis_title="Delivery Time (Days)",
    height=450,
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    legend=dict(title="Priority", orientation="h", yanchor="bottom", y=1.02)
)

```

)

```
st.plotly_chart(fig4, use_container_width=True)
```

```
# AI Insights for Service
```

```
st.markdown("#### AI Analysis")
```

```
service_insights = generate_service_insights(filtered_df)
```

```
st.markdown("""
```

```
<div class="info-box">
```

```
    <h4>Service Quality Insights</h4>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
for insight in service_insights:
```

```
    st.markdown(f'- {insight}')
```

```
st.markdown("---")
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    st.markdown("#### 🎯 Carrier Performance Matrix")
```

```
heatmap_df = filtered_df.groupby(['Carrier',  
'Delivery_Status']).size().reset_index(name='Order_Count')
```

```
heatmap_pivot = heatmap_df.pivot(index='Carrier', columns='Delivery_Status',  
values='Order_Count').fillna(0)
```

```
fig5 = go.Figure(data=go.Heatmap(
```

```

z=heatmap_pivot.values,
x=heatmap_pivot.columns,
y=heatmap_pivot.index,
colorscale='RdYlGn_r',
text=heatmap_pivot.values,
texttemplate='%{text:.0f}',
textfont={"size": 12},
colorbar=dict(title="Orders")
))

```

```

fig5.update_layout(
    title="",
    xaxis_title="Delivery Status",
    yaxis_title="Carrier",
    height=400
)

```

```

st.plotly_chart(fig5, use_container_width=True)

```

```

# Performance Score

```

```

st.markdown("##### 🇮🇹 Carrier Performance Score")

```

```

carrier_score = filtered_df.groupby('Carrier').agg({
    'Delivery_Status': lambda x: (x == 'On-Time').sum() / len(x) * 100,
    'Rating': 'mean'
}).reset_index()

carrier_score.columns = ['Carrier', 'On_Time_Pct', 'Avg_Rating']

carrier_score['Performance_Score'] = (carrier_score['On_Time_Pct'] * 0.6 +
                                      carrier_score['Avg_Rating'] * 20 * 0.4)

carrier_score = carrier_score.sort_values('Performance_Score', ascending=False)

```

```

for idx, row in carrier_score.iterrows():

```

```

score_color = COLORS['success'] if row['Performance_Score'] >= 80 else \
    COLORS['warning'] if row['Performance_Score'] >= 60 else
COLORS['danger']

st.markdown(f"""
    <div style="background: linear-gradient(90deg, {score_color}20 0%,
{score_color}10 100%);
    padding: 10px; border-radius: 8px; margin: 5px 0; border-left: 4px solid
{score_color};">
        <strong>{row['Carrier']}</strong>: {row['Performance_Score']:.1f}/100
        (On-time: {row['On_Time_Pct']:.0f}%, Rating: {row['Avg_Rating']:.2f} ★)
    </div>
    """, unsafe_allow_html=True)

```

with col2:

```

st.markdown("#### 🔍 Root Cause Analysis")

```

```

sunburst_df = filtered_df.groupby(['Carrier', 'Delivery_Status',
'Issue_Category']).size().reset_index(name='Count')

```

```

fig6 = px.sunburst(
    sunburst_df,
    path=['Carrier', 'Delivery_Status', 'Issue_Category'],
    values='Count',
    color='Count',
    color_continuous_scale='Reds',
    title=""
)

```

```

fig6.update_layout(
    height=500,
    margin=dict(t=10, l=0, r=0, b=0)
)

```

```

st.plotly_chart(fig6, use_container_width=True)

# Top Issues

st.markdown("##### 🚨 Top Customer Issues")

issue_counts = filtered_df[filtered_df['Issue_Category'] !=
'No_Issue_Reported']['Issue_Category'].value_counts()

for issue, count in issue_counts.head(5).items():

    pct = count / len(filtered_df) * 100

    st.markdown(f"""

        <div style="background: #fef3c7; padding: 8px; border-radius: 6px; margin: 5px 0;
color: #78350f;">

            <strong>{issue}</strong>: {count} orders ({pct:.1f}%)

        </div>

        """, unsafe_allow_html=True)

with tab4:

    st.markdown("### Fleet Health & Asset Performance Intelligence")

    st.markdown("*Optimize fleet utilization and identify renewal opportunities*")

# Fleet KPIs

st.markdown("##### 📊 Fleet Performance Indicators")

avg_age = fleet_df['Age_Years'].mean()

avg_efficiency = fleet_df['Fuel_Efficiency_KM_per_L'].mean()

availability_pct = (fleet_df['Status'].value_counts(normalize=True).get('Available', 0)) *
100

maintenance_pct = (fleet_df['Status'].value_counts(normalize=True).get('Maintenance',
0)) * 100

col1, col2, col3 = st.columns(3)

```

with col1:

```
# Age Gauge

age_color = COLORS['success'] if avg_age < 5 else COLORS['warning'] if avg_age <
7 else COLORS['danger']
```

```
fig7_age = go.Figure(go.Indicator(
    mode="gauge+number+delta",
    value=avg_age,
    delta={'reference': 5, 'increasing': {'color': COLORS['danger']}},
    title={'text': "Average Fleet Age (Years)", 'font': {'size': 16}},
    gauge={
        'axis': {'range': [0, 10], 'tickwidth': 1},
        'bar': {'color': age_color, 'thickness': 0.7},
        'steps': [
            {'range': [0, 5], 'color': '#d1fae5'},
            {'range': [5, 7], 'color': '#fef3c7'},
            {'range': [7, 10], 'color': '#fee2e2'}
        ],
        'threshold': {
            'line': {'color': "red", 'width': 4},
            'thickness': 0.75,
            'value': 7
        }
    }
))
```

```
fig7_age.update_layout(height=280, margin=dict(t=60, b=20))
st.plotly_chart(fig7_age, use_container_width=True)
```

with col2:

```

# Efficiency Gauge

eff_color = COLORS['success'] if avg_efficiency > 8 else COLORS['warning'] if
avg_efficiency > 6 else COLORS['danger']

```

```

fig7_eff = go.Figure(go.Indicator(
    mode="gauge+number+delta",
    value=avg_efficiency,
    delta={'reference': 8, 'decreasing': {'color': COLORS['danger']}},
    number={'suffix': " km/L"},
    title={'text': "Average Fuel Efficiency", 'font': {'size': 16}},
    gauge={
        'axis': {'range': [4, 12], 'tickwidth': 1},
        'bar': {'color': eff_color, 'thickness': 0.7},
        'steps': [
            {'range': [4, 6], 'color': '#fee2e2'},
            {'range': [6, 8], 'color': '#fef3c7'},
            {'range': [8, 12], 'color': '#d1fae5'}
        ],
        'threshold': {
            'line': {'color': "green", 'width': 4},
            'thickness': 0.75,
            'value': 8
        }
    }
))

```

```

fig7_eff.update_layout(height=280, margin=dict(t=60, b=20))

st.plotly_chart(fig7_eff, use_container_width=True)

```

with col3:

```

# Availability Gauge

```

```
    avail_color = COLORS['success'] if availability_pct > 80 else COLORS['warning'] if
availability_pct > 60 else COLORS['danger']
```

```
fig7_avail = go.Figure(go.Indicator(
    mode="gauge+number+delta",
    value=availability_pct,
    delta={'reference': 80, 'decreasing': {'color': COLORS['danger']}},
    number={'suffix': "%"},
    title={'text': "Fleet Availability", 'font': {'size': 16}},
    gauge={
        'axis': {'range': [0, 100], 'tickwidth': 1},
        'bar': {'color': avail_color, 'thickness': 0.7},
        'steps': [
            {'range': [0, 60], 'color': '#fee2e2'},
            {'range': [60, 80], 'color': '#fef3c7'},
            {'range': [80, 100], 'color': '#d1fae5'}
        ],
        'threshold': {
            'line': {'color': "green", 'width': 4},
            'thickness': 0.75,
            'value': 80
        }
    }
))
```

```
fig7_avail.update_layout(height=280, margin=dict(t=60, b=20))
```

```
st.plotly_chart(fig7_avail, use_container_width=True)
```

```
# AI Insights for Fleet
```

```
st.markdown("#### AI Analysis")
```

```
fleet_insights = generate_fleet_insights(fleet_df)
```

```
st.markdown("""  
<div class="info-box">  
    <h4>Fleet Optimization Insights</h4>  
</div>  
""", unsafe_allow_html=True)
```

```
for insight in fleet_insights:
```

```
    st.markdown(f'- {insight}')
```

```
st.markdown("---")
```

```
st.markdown("####  Fleet Efficiency Trends")
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    fig8 = px.scatter(  
        fleet_df,  
        x='Age_Years',  
        y='Fuel_Efficiency_KM_per_L',  
        color='Vehicle_Type',  
        size='CO2_Emissions_Kg_per_KM',  
        trendline='ols',  
        title="Age vs. Fuel Efficiency",  
        color_discrete_sequence=px.colors.qualitative.Bold  
    )
```

```
fig8.update_layout(  
    height=400,  
    xaxis_title="Vehicle Age (Years)",
```

```

        yaxis_title="Fuel Efficiency (km/L)",
        plot_bgcolor='rgba(0,0,0,0)',
        paper_bgcolor='rgba(0,0,0,0)'
    )

```

```

st.plotly_chart(fig8, use_container_width=True)

```

with col2:

```

fig9 = px.scatter(
    fleet_df,
    x='Age_Years',
    y='CO2_Emissions_Kg_per_KM',
    color='Vehicle_Type',
    size='Fuel_Efficiency_KM_per_L',
    trendline='ols',
    title="Age vs. CO2 Emissions",
    color_discrete_sequence=px.colors.qualitative.Bold # FIXED typo
)

```

```

fig9.update_layout(
    height=400,
    xaxis_title="Vehicle Age (Years)",
    yaxis_title="CO2 Emissions (kg/km)",
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)'
)

```

```

st.plotly_chart(fig9, use_container_width=True)

```

Fleet Composition

```

st.markdown("#### 🚗 Fleet Composition Analysis")

```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    fleet_type_dist = fleet_df['Vehicle_Type'].value_counts()
```

```
    fig_fleet_type = go.Figure(data=[go.Pie(
        labels=fleet_type_dist.index,
        values=fleet_type_dist.values,
        hole=0.4,
        marker_colors=[COLORS['primary'], COLORS['secondary'], COLORS['info']]
    )])
```

```
    fig_fleet_type.update_layout(
        title="Fleet Distribution by Vehicle Type",
        height=350
    )
```

```
st.plotly_chart(fig_fleet_type, use_container_width=True)
```

```
with col2:
```

```
    fleet_status_dist = fleet_df['Status'].value_counts()
```

```
    # Determine colors based on available statuses
```

```
    status_colors = []
```

```
    for status in fleet_status_dist.index:
```

```
        if status == 'Available':
```

```
            status_colors.append(COLORS['success'])
```

```
        elif status == 'Maintenance':
```

```
            status_colors.append(COLORS['warning'])
```

```
        else:
```

```
status_colors.append(COLORS['danger'])
```

```
fig_fleet_status = go.Figure(data=[go.Pie(  
    labels=fleet_status_dist.index,  
    values=fleet_status_dist.values,  
    hole=0.4,  
    marker_colors=status_colors  
)])
```

```
fig_fleet_status.update_layout(  
    title="Fleet Status Distribution",  
    height=350  
)
```

```
st.plotly_chart(fig_fleet_status, use_container_width=True)
```

```
with tab5:
```

```
st.markdown("### 📄 Data Explorer & Export")
```

```
st.markdown("*Download filtered data for further analysis*")
```

```
# Summary Statistics
```

```
col1, col2, col3 = st.columns(3)
```

```
with col1:
```

```
st.markdown(f"""
```

```
<div class="info-box">
```

```
<h4> 📊 Dataset Summary</h4>
```

```
<p><strong>Filtered Records:</strong> {len(filtered_df):} (out of {len(master_df):}  
total)</p>
```

```
<p><strong>Date Range:</strong> {filtered_df['Order_Date'].min().date()} to  
{filtered_df['Order_Date'].max().date()}</p>
```

```
<p><strong>Unique Carriers:</strong> {filtered_df['Carrier'].nunique()}</p>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
with col2:
```

```
st.markdown(f"""
```

```
<div class="success-box">
```

```
<h4> 💰 Financial Summary</h4>
```

```
<p><strong>Total Revenue:</strong>
₹{filtered_df['Order_Value_INR'].sum():.0f}</p>
```

```
<p><strong>Total Profit:</strong>
₹{filtered_df['Adjusted_Profit_INR'].sum():.0f}</p>
```

```
<p><strong>Avg. Margin:</strong>
{filtered_df['Profit_Margin_Pct'].mean():.2f}%</p>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
with col3:
```

```
st.markdown(f"""
```

```
<div class="warning-box">
```

```
<h4> 🌱 Sustainability Summary</h4>
```

```
<p><strong>Total Emissions:</strong> {filtered_df['Total_CO2_kg'].sum():.0f}
kg</p>
```

```
<p><strong>Avg. per Order:</strong> {filtered_df['Total_CO2_kg'].mean():.2f}
kg</p>
```

```
<p><strong>Carbon Cost:</strong> ₹{(filtered_df['Total_CO2_kg'] *
carbon_cost_inr_per_kg).sum():.0f}</p>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
st.markdown("---")
```

```
# Export Options
```

```
excel_data = to_excel(filtered_df)
```

```
try:
```

```
    start_date_str = date_range[0].strftime('%Y%m%d')
```

```
    end_date_str = date_range[1].strftime('%Y%m%d')
```

```
    file_name = f'logistics_control_tower_{start_date_str}_to_{end_date_str}.xlsx'
```

```
except Exception:
```

```
    file_name = "logistics_control_tower_data.xlsx"
```

```
col1, col2 = st.columns([2, 1])
```

```
with col1:
```

```
    st.download_button(
```

```
        label="📄 Download Full Dataset (Excel)",
```

```
        data=excel_data,
```

```
        file_name=file_name,
```

```
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
```

```
        use_container_width=True
```

```
    )
```

```
with col2:
```

```
    st.markdown(f'***Records to export:** {len(filtered_df):,}')
```

```
st.markdown("---")
```

```
st.markdown("#### 🔍 Data Preview")
```

```
# Display options
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    show_rows = st.slider("Number of rows to display", 10, 500, 100, 10)
```

```
with col2:
```

```

column_filter = st.multiselect(
    "Select columns to display",
    options=filtered_df.columns.tolist(),
    default=['Order_ID', 'Order_Date', 'Carrier', 'Origin', 'Destination',
            'Product_Category', 'Adjusted_Profit_INR', 'Total_CO2_kg',
            'Delivery_Status', 'Rating']
)

if column_filter:
    st.dataframe(
        filtered_df[column_filter].head(show_rows),
        use_container_width=True,
        height=400
    )
else:
    st.dataframe(
        filtered_df.head(show_rows),
        use_container_width=True,
        height=400
    )

# Footer
st.markdown("---")
st.markdown("""
<div style='text-align: center; color: #64748b; padding: 20px;'>
    <p><strong>NexGen Logistics Control Tower</strong> | Powered by Advanced Analytics
    & AI</p>
</div>
""", unsafe_allow_html=True)

if __name__ == "__main__":

```

main()

requirements.txt:

streamlit
pandas
plotly
xlsxwriter
pyngrok

readme.md:

NexGen Logistics Control Tower

1. Problem Statement: The Logistics Trilemma

Modern logistics operates under the pressure of optimizing three often conflicting goals: maximizing Profitability, ensuring Sustainability, and delivering excellent Performance. Traditional tools analyze these in silos, creating critical blind spots:

- Hidden Costs: Profit calculations ignore warehouse storage costs, misrepresenting true end-to-end profitability.
- Disconnected Sustainability: Aggregate CO₂ data is not linked to per-order profitability or fleet health.
- Opaque Performance Issues: Customer feedback isn't tied back to specific carriers, routes, or operational failures.

The result is suboptimal trade-offs, forming what we call the "Logistics Trilemma."

2. Solution: The Logistics Control Tower

The Logistics Control Tower is an interactive Streamlit dashboard built with Python (Pandas and Plotly) that unifies all seven NexGen datasets into a single, integrated view.

Key Innovations:

- True End-to-End Profitability (E2E_Profit_INR)
- Profit vs. Sustainability Quadrant analysis
- Carbon Tax Simulator (₹0–₹50 per kg CO₂)
- AI-Generated Actionable Insights
- Advanced Visualizations including Treemaps, Sunbursts, Gauges, and Heatmaps
- Modern UI/UX with a clean, professional, and responsive design

This dashboard empowers managers and analysts to optimize all three pillars profitability, sustainability, and service quality simultaneously.

3. Key Features

- Holistic Data Integration combining all 7 CSVs: orders, delivery_performance, cost_breakdown, routes_distance, customer_feedback, warehouse_inventory, and vehicle_fleet.
- Strategic Quadrant Analysis classifies orders into "Best-in-Class," "Optimize for Green," "Scale Up," and "Critical Intervention" categories.
- Profitability Deep Dive through violin plots, treemaps, and carrier margin comparisons.
- Service Excellence Dashboard with reliability heatmaps, delivery speed analysis, and customer sentiment mapping.
- Fleet Intelligence Module featuring health gauges, scatter plots, and composition charts.
- Dynamic AI Insights providing context-aware recommendations.
- What-If Carbon Tax Simulation with interactive profit adjustments.
- Data Explorer and Export allowing preview and download of filtered data as Excel files.

4. Project Structure

```
NexGen_Logistics_Control_Tower/
|
|— app.py           # Main Streamlit application
|— requirements.txt # Python dependencies
|— README.md       # This file
|— Innovation_Brief.pdf # Business case & technical report
|
|— cost_breakdown.csv # REQUIRED: All 7 data files in this folder
|— customer_feedback.csv
|— delivery_performance.csv
|— orders.csv
|— routes_distance.csv
|— vehicle_fleet.csv
|— warehouse_inventory.csv
```

5. Setup and Usage (Running on Localhost)

Prerequisites:

- Python 3.9 or higher
- All seven CSVs placed in the same directory as app.py

Steps:

1. Create and enter the project folder.

Command: `mkdir NexGen_Logistics_Control_Tower`

`cd NexGen_Logistics_Control_Tower`

2. Clone the repository or add project files

Command: git clone <https://github.com/chopra-mayank/Case-Study-OFI-Services.git>

3. Create a virtual environment Command: `python -m venv venv`
4. Activate the virtual environment
macOS / Linux: `source venv/bin/activate`
Windows: `venv\Scripts\activate`
5. Install dependencies Command: `pip install -r requirements.txt`
6. Run the Streamlit app Command: `streamlit run app.py`

Access the dashboard at <http://localhost:8501>

6. How It Works

- Tabs: Navigate between strategic, profitability, service, and fleet views.
- Sidebar Filters: Adjust by date, carrier, segment, and other dimensions.
- Interactivity: Hover, zoom, or filter within Plotly charts.
- AI Insights: Get real-time, context-based recommendations.
- Data Explorer: Download filtered data snapshots for offline analysis.

7. Technical Stack and KPIs

Core Stack:

Python 3.9+

Streamlit

Pandas

NumPy

Plotly Styling: Custom CSS for a professional interface

Key KPIs:

- E2E_Profit_INR, Adjusted_Profit_INR
- Total_CO2_kg, CO2_per_KM
- Carrier_Performance_Score, Cost_per_KM

8. Data Requirements

The following seven CSV files must be placed in the project root directory:

1. orders.csv
2. delivery_performance.csv
3. cost_breakdown.csv
4. routes_distance.csv

5. `customer_feedback.csv`
6. `vehicle_fleet.csv`
7. `warehouse_inventory.csv`

For detailed data schema, refer to the `Mayank_Chopra.pdf` file.

9. Troubleshooting

- `FileNotFoundError`: Ensure all CSVs are located in the same folder as `app.py`
- `ModuleNotFoundError`: Run `pip install -r requirements.txt`
- No Data Displayed: Loosen sidebar filters
- Slow Load Times: Use smaller date ranges or restart the app (Ctrl+C, then rerun)
- Chart Display Issues: Perform a hard browser refresh